

Black Vault

Category: Forensics

Subcategory: Linux Memory Forensics

Difficulty: *Insane*

File: *memdump.raw*

Flag: *OBV{NULL_ROUTE_0xB17}*

Objective

Analyze the Linux memory dump, identify the hidden Black Vault memory regions, derive the XOR keys using the known **BV1** : prefix, reconstruct the scattered fragments in the correct order, and recover the hidden flag.

Intended Solve

1. Identify the artifact

Start by checking the memory dump: **file memdump.raw**

It is an ELF core dump, so this is a Linux memory-forensics challenge.

2. Search for useful forensic clues

Since the only initial clue is the flag format **OBV{}**, start with readable strings:

strings -a memdump.raw | grep -E 'BVLT|OBV|vault|black'

The important result is:

BVLT|VERSION=9|REGIONS=4|ORDER=R1,R2,R3,R4|TRANSFORM=XOR8|KNOWN_PREFIX_DERIVES_KEY|

This is the main forensic clue.

It tells us:

- There are 4 regions
- The order is **R1** → **R2** → **R3** → **R4**
- The transformation is XOR8
- The known plaintext prefix is **BV1** :
- Each region has a specified offset and size
- The key can be derived from the known prefix

3. Inspect the metadata area

Locate the metadata:

```
grep -aobF 'BVLT|VERSION=9' memdump.raw
```

Then inspect around it:

```
dd if=memdump.raw bs=1 skip=$((0x1AC400)) count=512 2>/dev/null | xxd -g 1
```

The metadata is followed by encrypted-looking bytes:

```
73 67 00 0b 7e 73 67 4a 7f 64 7d
```

The metadata specifies:

```
R1:128:11
```

So this 11-byte sequence is the R1 ciphertext.

4. Understand the XOR clue

The metadata says:

```
KNOWN=BV1:
```

```
HINT=KNOWN_PREFIX_DERIVES_KEY
```

Therefore we know the first four plaintext bytes are:

```
BV1:
```

ASCII hexadecimal:

```
42 56 31 3a
```

R1 starts with:

73 67 00 0b

XOR the known plaintext against the ciphertext:

73 XOR 42 = 31

67 XOR 56 = 31

00 XOR 31 = 31

0b XOR 3a = 31

Therefore:

R1 XOR key = 0x31

5. Decode R1

Run this command :

```
python3 - <<'PY'
```

```
cipher = bytes.fromhex("7367000b7e73674a7f647d")
```

```
key = 0x31
```

```
print(bytes(b ^ key for b in cipher))
```

```
PY
```

Result :

```
b'BV1:OBV{NUL'
```

Remove the **BV1 :** marker:

```
OBV{NUL
```

6. Locate the remaining fragments

Search the dump for the other XOR-consistent **BV1 :** fragments:

```
python3 - <<'PY'
```

```
from pathlib import Path
```

```
data = Path("memdump.raw").read_bytes()
```

```
for name, size in [
```

```
    ("R2", 9),
```

```
    ("R3", 9),
```

```
    ("R4", 8),
```

```
]:
```

```
    print(f"\n{name} candidates:")
```

```
    for pos in range(0, len(data) - size):
```

```
        chunk = data[pos:pos + size]
```

```
        key = chunk[0] ^ ord('B')
```

```
        if all((chunk[i] ^ key) == b
```

```
            for i, b in enumerate(b"BV1:")):
```

```
                decoded = bytes(b ^ key for b in chunk)
```

```
if b"OBV" in decoded or decoded.startswith(b"BV1:"):

```

```
    print(
        f"0x{pos:X} "
        f"key=0x{key:02X} "
        f"{decoded!r}"
    )

```

PY

The meaningful fragments are:

R1 → 0x1AC4B0 → key 0x31 → BV1:OBV{NUL

R2 → 0x1AB530 → key 0x62 → BV1:L_ROU

R3 → 0x1AA5B0 → key 0xA4 → BV1:TE_0x

R4 → 0x1A9630 → key 0xD7 → BV1:B17}

The other matches are decoys/noise.

7. Decode R2

```
python3 - <<'PY'

```

```
cipher = bytes.fromhex("203453582e3d302d37")

```

```
key = 0x62

```

```
print(bytes(b ^ key for b in cipher))

```

PY

Result: **BV1:L_ROU**

Fragment: **L_ROU**

8. Decode R3

```
python3 - <<'PY'  
  
cipher = bytes.fromhex("e6f2959ef0e1fb94dc")  
  
key = 0xA4  
  
print(bytes(b ^ key for b in cipher))  
  
PY
```

Result: **BV1:TE_0x**

Fragment: **TE_0x**

9. Decode R4

```
python3 - <<'PY'  
  
cipher = bytes.fromhex("9581e6ed95e6e0aa")  
  
key = 0xD7  
  
print(bytes(b ^ key for b in cipher))  
  
PY
```

Result: **BV1:B17}**

Fragment: **B17}**

10. Reconstruct the flag

The metadata specifies:

ORDER=R1,R2,R3,R4

Therefore:

R1 = OBV{NUL

R2 = L_ROU

R3 = TE_0x

R4 = B17}

Concatenate them:

OBV{NUL + L_ROU + TE_0x + B17}

Flag

OBV{NULL_ROUTE_0xB17}